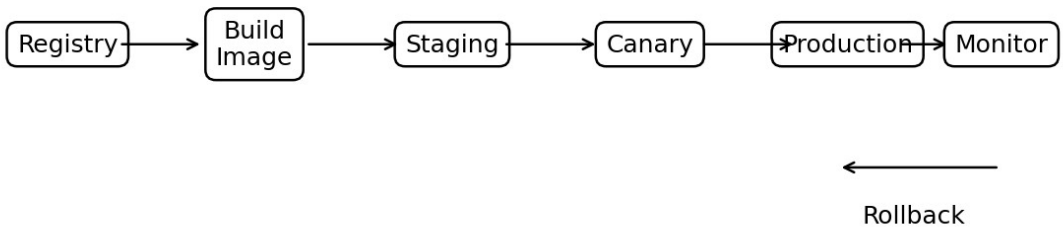
42. DEPLOYMENT

Batch • Online Serving • Cloud • Canary • Autoscaling • Rollback • Production Readiness

ML deployment yaşam döngüsü



Deployment = artifact'i kontrollü biçimde çalışan servise dönüştürme

Bu modülün amacı
Registry'de hazır olan bir modeli gerçek kullanıcı veya iş akışına güvenli biçimde taşımak. Batch scoring, online API, async/serverless inference, container registry, managed endpoints, canary/blue-green rollout, autoscaling, rollback, observability ve maliyet kararları veri bilimci perspektifinden ele alınacak.

Kazanım	Modül sonunda
Mode seçimi	Batch, online, async ve serverless serving arasında doğru kararı verebilirsin.
Release	Staging, canary, blue-green ve shadow rollout desenlerini anlayabilirsin.
Scaling	Replica, worker, batching ve autoscaling kapasite kararlarını kurabilirsin.
Cloud	Container registry, managed endpoint ve serverless deployment mantığını kavrayabilirsin.
Reliability	Healthcheck, timeout, retry, rollback ve fallback tasarlayabilirsin.
Operations	Latency, throughput, error rate, cost ve model sinyallerini production'da takip edebilirsin.

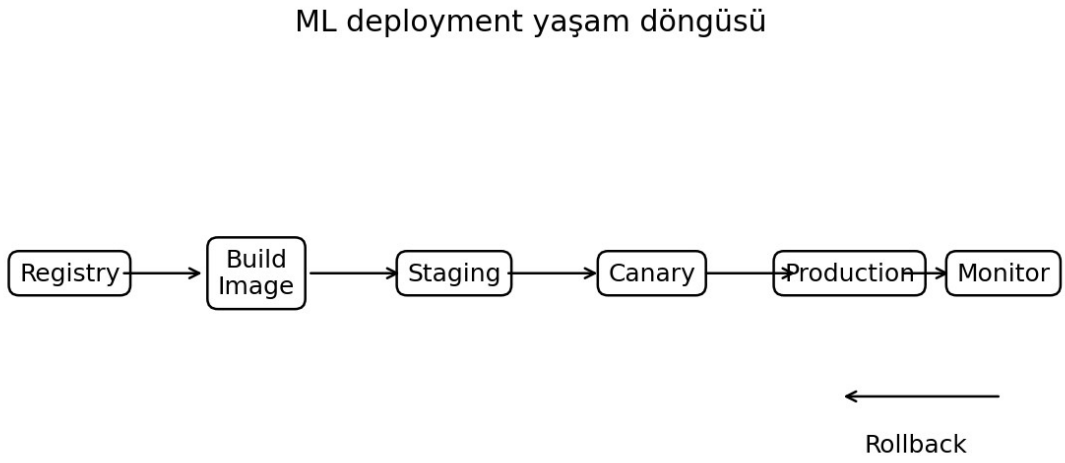
1. Deployment nedir?

Deployment, geliştirilmiş ve doğrulanmış model/artifact'in gerçek iş yükünde çalışacak bir runtime'a güvenli biçimde taşınmasıdır.

Artifact	Deployment
Model dosyası	Çalışan service/job
Offline metric	Production SLA
Local notebook	Managed runtime
Tek kullanıcı	Concurrent workload
Manuel çağrı	Automated traffic/jobs

Production gerçeği
İyi model + kötü deployment = kötü ürün. Kullanıcı sadece tahmini değil, gecikme ve hata deneyimini de yaşar.

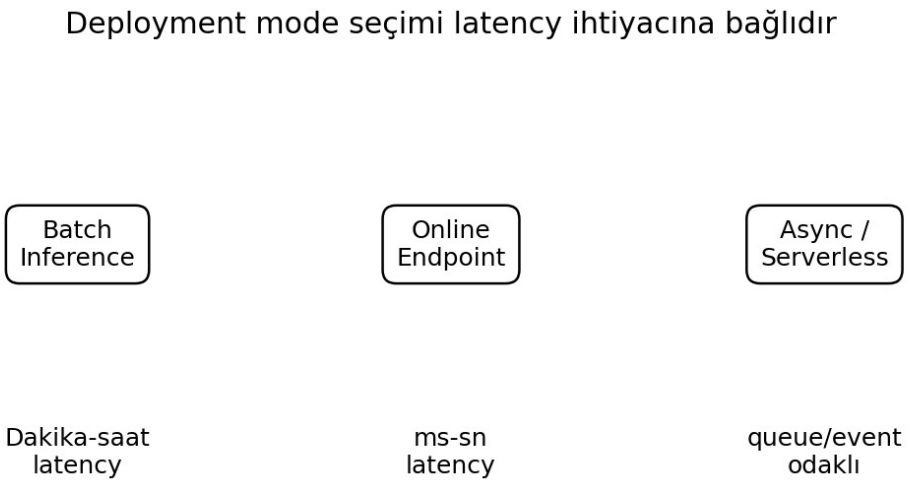
2. Deployment lifecycle



- Registry'den candidate seç
- Serving image/artifact oluştur
- Staging deploy et
- Smoke/integration test
- Controlled rollout
- Monitor
- Gerekirse rollback

Deployment bir an değil
Release, verification ve monitoring içeren kontrollü süreçtir.

3. Serving mode seçimi



Mode	Latency	Tipik kullanım
Batch	Dakika-saat	Churn scores, daily recommendations
Online	Milisaniye-saniye	Fraud, ranking, realtime decisions
Async	Saniye-dakika	Document/image inference
Serverless	Değişken	Düşük/seyrek trafik, event-driven

4. Batch inference

Model belirli bir dataset üzerinde toplu olarak çalıştırılır ve output tablo/dosyaya yazılır.

```
features = read_features(run_date)
model = load_model("churn_model", version=17)

scores = model.predict_proba(features)[:, 1]

write_scores(
  run_date=run_date,
  scores=scores
)
```

Avantaj
Basit, yüksek throughput, düşük operational complexity. Realtime gerekmiyorsa çoğu use case için en doğru başlangıç olabilir.

5. Batch SLA

SLA boyutu	Örnek
Start	02:00

Complete by	04:00
Freshness	T-1 data
Rows	10M müşteri
Failure	Retry + alert

Availability yerine timeliness
Batch sistemde ana soru API uptime değil, job doğru zamanda tamamlandı mı?

6. Incremental batch scoring

Tüm evreni tekrar score etmek yerine yalnız değişen/yeni entity'leri score etme.

Trade-off
Daha ucuz olabilir ama hangi kayıtların yeniden score edilmesi gerektiğini doğru tespit etmek gerekir.

7. Online inference

Request geldiğinde düşük latency ile model prediction üretir.

Metric	Online serving
p50/p95/p99	Latency
RPS/QPS	Throughput
Error rate	Reliability
Availability	Uptime
Cost/request	Efficiency

8. Online request path

```
client
  -> gateway
  -> feature lookup
  -> preprocessing
  -> model inference
  -> postprocessing
  -> response
```

Latency budget
Toplam SLA tüm adımların toplamıdır. 100 ms hedefte feature lookup 80 ms ise model optimize etmek tek başına çözüm değildir.

9. Stateless service

Online prediction service mümkün olduğunca request bazında stateless tasarlanır.

Neden?
Replica sayısını artırmak, restart etmek ve traffic dağıtmak kolaylaşır.

10. Stateful dependency

- Online feature store
- Database
- Cache
- Vector DB

State'i servis dışına taşı
Model API'nin kendi local disk/memory'sinde kullanıcı state tutmak scaling ve failover'ı zorlaştırır.

11. Async inference

Client request'i bırakır; iş queue'ya alınır, sonuç daha sonra polling/webhook/object storage ile alınır.

Uygun use case	Neden
Video inference	Uzun işlem
Large document OCR/NLP	Saniyeler-dakikalar
Bulk embeddings	Queue/batch faydası
Heavy LLM generation	Timeout riski

12. Queue-based architecture

```
API
  -> Queue
  -> Worker
  -> Result Store
  -> Callback / Polling
```

Backpressure
Queue, gelen yükü worker kapasitesinden ayırır ve burst traffic'i emer.

13. Serverless inference

Infrastructure yönetmeden request/event geldikçe çalışan compute modeli.

Artı	Eksi
Scale-to-zero	Cold start
Usage-based cost	Runtime/memory sınırları
Ops düşük	Büyük model yükleme maliyeti
Event integration kolay	Platform bağımlılığı

İyi aday
Düşük trafikli küçük/orta model endpoint'i veya event-driven batch task.

14. Cold start

Yeni instance başlatılırken runtime, dependencies ve model yüklenmesi nedeniyle ilk request'in yavaş olması.

Azaltma
Warm instances, küçük image, hızlı model loading, preloading veya provisioned concurrency.

15. Managed endpoints

Cloud ML platformlarının autoscaling, TLS, metrics ve rollout gibi serving altyapısını yönettiği endpoint hizmetleri.

- Managed model hosting
- Autoscaling
- Traffic splitting
- Logging/metrics

Trade-off
Daha az ops, daha yüksek platform bağımlılığı ve bazen maliyet.

16. Container registry

Serving image'larının merkezi ve versioned olarak saklandığı registry.

```
registry.example.com/ml/churn-api:git-abc123
```

Tag + digest
Human-readable tag ve immutable digest birlikte kullanılır.

17. Image ve model versiyon ilişkisi

Service image	Model version
churn-api:abc123	churn_model:17
fraud-api:def456	fraud_model:42

Lineage
Deployment kaydı service image digest + model version + config version içermelidir.

18. Staging environment

Production'a benzer ama gerçek kullanıcıyı etkilemeyen ortam.

- Smoke test
- Schema test
- Load test
- Dependency connectivity
- Latency benchmark

Aynı olmak zorunda değil
Ama kritik runtime/version/network farkları production sürprizi yaratmayacak kadar yakın olmalı.

19. Smoke test

```
curl -X POST   http://staging/predict   -H "Content-Type: application/json"   -d '{"age":42,"income":85000}'
```

Amaç
Servis açılıyor mu, model yüklenmiş mi, endpoint temel happy-path request'i işliyor mu?

20. Integration test

Service + gerçek dependency'lerin birlikte davranışını test eder.

- Feature store
- Database
- Model artifact
- Auth

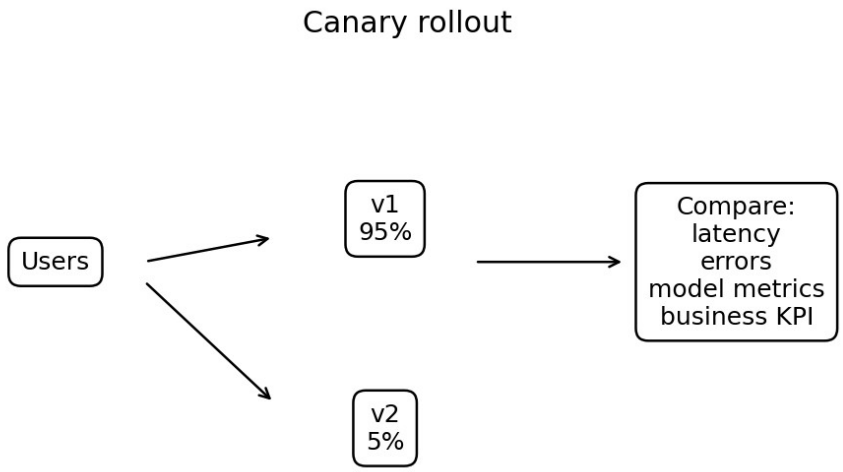
Production-like data değil
Sensitive gerçek müşteri datası yerine kontrollü fixture/test tenant kullanılmalı.

21. Contract test

Request/response schema'nın client beklentisini bozmadığını doğrular.

Backward compatibility
Yeni model release API contract'ını istemeden değiştirmemeli.

22. Canary deployment



- %1-5 trafik
- Latency/error comparison
- Prediction drift
- Business KPI

Amaç
Tüm kullanıcıları riske atmadan production doğrulaması.

23. Canary gate

Metric	Örnek gate
Error rate	v2 <= v1 + 0.2%
p95 latency	< 120 ms
Positive rate	beklenen band
Business KPI	negative regression yok

24. Blue/Green deployment

Blue mevcut production, green yeni release. İki ortam yan yana hazır tutulur ve traffic switch yapılır.

Rollback
Green sorunluysa traffic blue'ya hızlıca döner.

25. Rolling deployment

Replica'lar sırayla yeni versiyona güncellenir.

Risk
Bir süre iki version aynı anda çalışır; backward-compatible request/response ve shared state gerekir.

26. Shadow deployment

Production request'lerinin kopyası yeni modele gönderilir ama cevabı kullanıcıyı etkilemez.

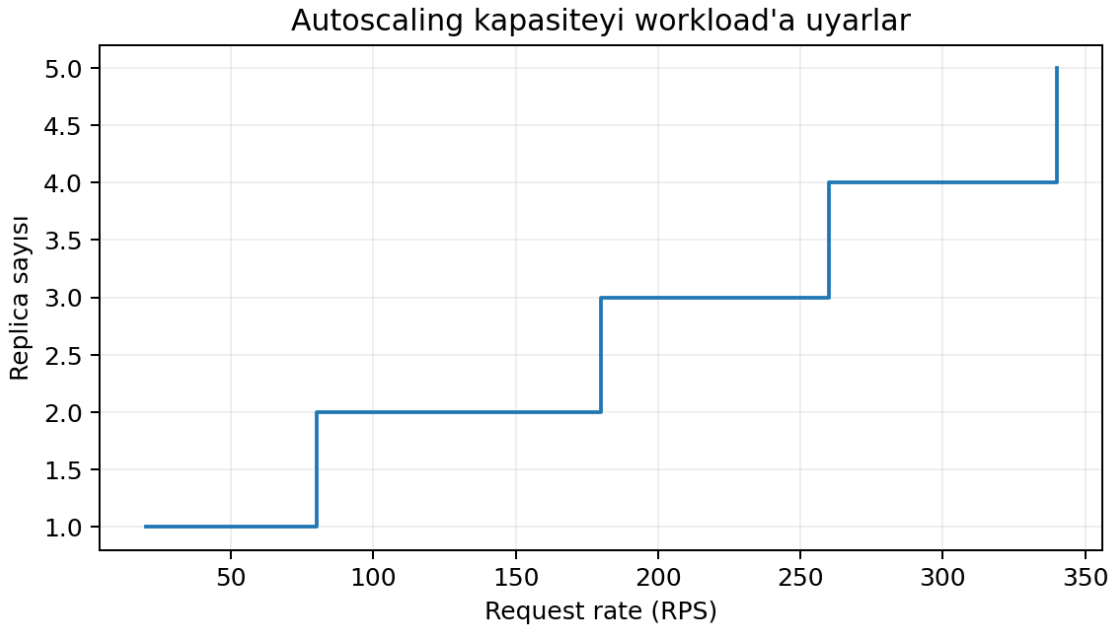
Güçlü kullanım
Gerçek traffic distribution üzerinde latency ve output behavior ölçmek.

27. A/B testing deployment

Kullanıcılar sabit deney gruplarına ayrılır; model etkisi business metric ile ölçülür.

Canary ile fark
Canary release riskini ölçer; A/B test nedensel business etkiyi ölçmeye odaklanır.

28. Autoscaling



Scaling signal	Örnek
CPU/GPU	Utilization
Traffic	RPS
Queue	Pending jobs
Latency	p95 threshold

Custom	Tokens/sec

Doğru sinyal
GPU model serving'de CPU düşük olsa da GPU %100 olabilir; autoscaling metriği workload'a uygun seçilmeli.

29. Horizontal vs Vertical scaling

Horizontal	Vertical
Daha çok replica	Daha büyük makine
Fault tolerance artar	Basit ama limitli
Traffic distribution gerekir	Restart gerekebilir

Serving default
Stateless online API için horizontal scaling genellikle daha esnektir.

30. Worker sayısı

Bir container/process içinde worker sayısı CPU concurrency'yi artırabilir.

Model memory
Her worker modeli ayrı yüklerse RAM/GPU memory çarpılır. Worker sayısını körlemesine artırma.

31. Dynamic batching

Yakın zamanda gelen request'leri küçük batch halinde tek model forward pass'te işlemek.

Artı	Eksi
GPU throughput artar	Queue latency eklenir
Daha iyi utilization	Tail latency kötüleşebilir

LLM/embedding
GPU workload'larında batching çok önemlidir.

32. Concurrency limit

Servisin aynı anda kaç request işleyebileceğini sınırlamak overload ve OOM riskini azaltır.

Queueing
Limit üstündeki request'ler queue, 429 veya backpressure mekanizmasına yönlendirilebilir.

33. Load testing

- RPS
- Concurrent users
- p95/p99
- Error rate
- CPU/GPU
- Memory

Kapasite kanıtı
Production peak tahminine dayalı load test olmadan instance sayısı yalnız tahmindir.

34. Capacity planning

```
safe_rps_per_replica = 75
peak_rps = 300

replicas = ceil(
  peak_rps / safe_rps_per_replica
)
```

Headroom
Failure, burst ve maintenance için %20-50 kapasite payı bırakılabilir.

35. Health checks

Check	Soru
Liveness	Process yaşıyor mu?
Readiness	Traffic almaya hazır mı?
Startup	Ağır model loading tamamlandı mı?

Yanlış readiness
Sadece HTTP process ayakta diye ready dönme; model ve kritik dependency gerçekten hazır olmalı.

36. Timeout budget

Her downstream call için ayrı timeout ve toplam request SLA olmalıdır.

```
total_budget = 150 ms
feature_lookup = 40 ms
model = 60 ms
postprocess = 10 ms
network_headroom = 40 ms
```

Infinite wait yok
Timeout olmayan dependency tüm worker'ları bloke edebilir.

37. Retries

Hata	Retry
Transient 503	Evet, sınırlı
Network timeout	Evet, idempotent ise
Validation 400	Hayır
Auth 401/403	Hayır
OOM/model bug	Hayır

Retry storm
Backoff + jitter olmadan incident daha da büyüyebilir.

38. Circuit breaker

Downstream sürekli hata veriyorsa geçici süre çağrıları durdurup sistemi koruma pattern'i.

Amaç
Failing dependency'ye sürekli yük bindirmemek ve hızlı fallback.

39. Fallback

Use case	Fallback
Recommendation	Popularity baseline
Fraud	Rule engine/manual review
Churn	Previous batch score
Search/RAG	Keyword search

Degrade gracefully
Tahmin yokken tüm business flow'un durması gerekmeyebilir.

40. Rollback

- Previous image
- Previous model version
- Previous config
- Traffic switch

Rollback yalnız artifact değil
Serving code ve model compatibility birlikte düşünülmelidir.

41. Rollback trigger

Sinyal	Örnek
Error rate	5xx artışı
Latency	p95 SLA ihlali
Model output	Positive rate anomali
Business	Conversion düşüşü
Data	Feature missing spike

42. Model artifact loading

Model startup'ta registry/object storage'dan indirilebilir veya image içinde bulunabilir.

Yaklaşım	Artı	Eksi
Image içinde	Atomic release	Büyük image
Startup download	Model bağımsız version	Startup gecikmesi
Separate model server	Bağımsız scaling	Mimari complexity

43. Warm-up

Model yüklenip runtime optimize olduktan sonra dummy inference ile ilk-request latency azaltılabilir.

GPU/JIT
Compilation/cache nedeniyle ilk inference anormal yavaş olabilir.

44. Model server

Triton/TorchServe/TF Serving benzeri dedicated serving katmanları batching, model repository ve GPU optimization sağlayabilir.

Ne zaman?
Yüksek throughput veya çok model serving ihtiyacında.

45. API gateway

- TLS
- Authentication
- Rate limiting
- Routing
- Request size limit

Separation
Model container authentication mantığının tamamını kendi içinde taşımak zorunda değildir.

46. Service mesh

Service-to-service traffic için mTLS, retries, routing ve observability gibi özellikleri merkezi katmana taşıyabilir.

Gereksiz karmaşıklık
Küçük sistemde service mesh çoğu zaman şart değildir.

47. Kubernetes mental model

Kavram	Deployment karşılığı
Pod	Container runtime unit
Deployment	Desired replica/version
Service	Stable network endpoint
Ingress	External routing
ConfigMap/Secret	Runtime config
HPA	Autoscaling

Ezber değil
Temel fikir desired state + self-healing + scaling.

48. Kubernetes readiness

Readiness false ise pod çalışmaya devam eder ama service traffic göndermez.

Model rollout
Yeni pod model load tamamlanmadan traffic almamalı.

49. Pod disruption

Rolling update veya node maintenance sırasında minimum replica availability korunmalıdır.

Availability
Tek replica kritik online servis için deployment sırasında downtime riski taşır.

50. Cloud deployment seçenekleri

Seçenek	Ne zaman?
VM	Tam kontrol / özel runtime
Container service	Standart API/service
Kubernetes	Complex multi-service / scale
Managed ML endpoint	ML odaklı düşük ops
Serverless	Seyrek trafik/event-driven
Batch job service	Scheduled/ephemeral scoring

51. Managed endpoint vs self-hosted

Managed endpoint	Self-hosted
Ops düşük	Kontrol yüksek
Fast setup	Customization yüksek
Platform cost	Infra/ops cost
Built-in rollout/metrics	Kendin kurarsın

Takım kapasitesi
Deployment kararı teknik performans kadar operational yetkinliğe bağlıdır.

52. Multi-region deployment

Latency, availability veya regulatory sebeplerle birden fazla region.

- Active-active
- Active-passive
- Global load balancing

State
Feature store/model version consistency region'lar arasında yönetilmelidir.

53. Disaster recovery

Kavram	Anlam
RPO	Kabul edilen veri kaybı
RTO	Kabul edilen recovery süresi

Model service
Registry ve deployment config başka region'da yeniden kurulabilir olmalı.

54. Configuration

- MODEL_VERSION
- LOG_LEVEL
- FEATURE_STORE_URL
- REQUEST_TIMEOUT
- MAX_BATCH_SIZE

Aynı image farklı config
Dev/staging/prod environment farkları code içine gömülmemeli.

55. Secrets

Database password, API key veya cloud credential image/config içinde plain text tutulmamalıdır.

Production
Secret manager, workload identity veya orchestrator secret mechanism kullan.

56. Observability

Sinyal	Örnek
Logs	Request/model version/error
Metrics	Latency/RPS/error rate
Traces	Gateway -> feature -> model
Model metrics	Score distribution
Business	Conversion/fraud catch

Deployment başarısı
Servis ayakta + model davranışı sağlıklı + business etkisi kabul edilebilir.

57. Request tracing

```
request_id
  -> gateway
  -> feature service
  -> model endpoint
  -> downstream action
```

Incident
Bir kullanıcı hatasını tek request_id ile uçtan uca inceleyebilirsin.

58. Deployment dashboard

- Current image/model version
- Replica count
- p95/p99 latency
- 5xx rate
- CPU/GPU memory
- Prediction rate
- Score distribution

Release annotation
Dashboard'da deploy zamanı işaretlenirse regression'ı release ile ilişkilendirmek kolaylaşır.

59. Cost metrics

Metric	Örnek
Cost/request	Online endpoint
Cost/1k predictions	Serving efficiency
GPU utilization	Hardware efficiency
Idle replica cost	Autoscaling waste
Batch cost/run	Scheduled scoring

Maliyet = deployment metriği
En düşük latency her zaman en iyi ekonomik karar değildir.

60. SLO / SLA

Kavram	Örnek
SLI	p95 latency
SLO	p95 < 120 ms, 99% requests
SLA	Müşteriye verilen söz/kontrat

Error budget
SLO ihlali toleransını ürün hızı ile reliability arasında yönetmek için kullanılabilir.

61. Production readiness review

1. Model registry'de approved.
2. Image immutable ve tarandı.
3. Staging testleri geçti.
4. Load test tamamlandı.
5. Readiness/liveness doğru.
6. Autoscaling config hazır.
7. Timeout/retry/fallback tanımlı.
8. Monitoring dashboard hazır.
9. Rollback tested.
10. Owner/on-call belli.

62. Common anti-patterns

- Production'a doğrudan notebook'tan model kopyalamak.
- latest image tag kullanmak.
- Tek replica kritik API çalıştırmak.
- Canary yapmadan full traffic geçirmek.
- Readiness yalnız 200 dönsün diye yazmak.
- Modeli her request'te yüklemek.
- Worker sayısını memory hesabı yapmadan artırmak.

- Load test olmadan autoscaling kurmak.
- Rollback planı olmadan release yapmak.
- Sadece CPU/RAM izleyip model output'u izlememek.

63. Gerçek vaka: churn batch deployment

Katman	Karar
Mode	Daily batch
Artifact	Registry churn:v17
Runtime	Ephemeral container job
Input	Gold customer features
Output	CRM scoring table
Failure	Retry + previous scores fallback
Monitor	Row count, score distribution, completion time

64. Gerçek vaka: fraud online deployment

İhtiyaç	Tasarım
Latency	p95 < 80 ms
Features	Online feature store
Runtime	Multiple replicas
Scale	RPS/GPU utilization
Release	Shadow -> canary
Fallback	Rules/manual review
Monitor	Recall, FP rate, latency

65. Gerçek vaka: recommendation deployment

Candidate generation ve ranking farklı servisler veya batch+online hibrit olabilir.

- Daily item/user embeddings
- ANN candidate service
- Online ranker
- Redis/cache

Hybrid deployment
Her ML bileşen aynı serving mode'u kullanmak zorunda değildir.

66. Gerçek vaka: RAG deployment

Adım	Deployment concern
Retriever	Vector DB latency
Reranker	CPU/GPU capacity
LLM	Token latency/cost
API	Streaming response
Fallback	Keyword/search
Monitor	Retrieval + answer + service metrics

End-to-end budget
Retrieval + rerank + generation toplam latency birlikte yönetilir.

67. LLM streaming

Tokenlar hazır oldukça client'a stream ederek perceived latency azaltılır.

TTFT
Time-to-first-token LLM UX için önemli serving metriğidir.

68. Mülakat soruları

Soru	Kısa cevap
Batch vs online inference?	Periyodik toplu scoring vs request-time prediction.
Async inference?	Queue üzerinden uzun-running prediction.
Serverless ne zaman?	Seyrek/event-driven trafik ve düşük ops ihtiyacı.
Canary deployment?	Küçük trafikle kontrollü release.
Blue/Green?	Eski ve yeni ortam paralel; traffic switch.
Shadow?	Yeni model traffic'i görür ama kararı etkilemez.
Autoscaling metric?	Workload'a uygun CPU/GPU/RPS/queue/latency.
Readiness ne?	Traffic almaya gerçek hazır olma.
Rollback?	Önceki güvenli image/model/config'e dönüş.
Cold start?	Yeni instance/model loading ilk request gecikmesi.
Dynamic batching?	Request'leri GPU forward pass için birleştirme.
p95 neden?	Tail latency kullanıcı deneyimini daha iyi yansıtır.
Circuit breaker?	Failing dependency çağrılarını geçici kesme.
SLO vs SLA?	İç hedef vs dış sözleşme.
Managed endpoint avantajı?	Daha az serving/infra ops.

69. Uygulama alıştırmaları

11. Bir use case için batch mi online mı seç ve gerekçelendir.
12. FastAPI model servisini Docker image olarak build et.
13. Staging smoke test script'i yaz.
14. Canary için 5 metric gate tanımla.
15. p95 < 100 ms hedefi için latency budget çıkar.
16. Safe RPS/replica ölçüp kapasite hesabı yap.
17. Readiness endpoint'ini model yüklenmesine bağla.
18. Fallback ve rollback planı yaz.
19. Load test çıktısından autoscaling policy öner.
20. Deployment dashboard'unda izleyeceğin 10 metriği seç.

70. Mini portföy projesi

Proje: Production Deployment of a Churn Prediction Service

Dosya / katman	İçerik
app/	FastAPI prediction service
Dockerfile	Serving image
deploy/staging/	Staging config
deploy/prod/	Production config
load_test/	Capacity benchmark
scripts/smoke_test.py	Deployment validation
monitoring/	Latency/error/model metrics
runbooks/rollback.md	Rollback procedure
README.md	Architecture + SLO + rollout strategy

CV cümlesi
Deployed a containerized ML prediction service with staging validation, canary rollout, autoscaling, health checks, latency SLOs, structured monitoring, rollback procedures and production capacity testing.

71. Final cheat sheet

Kavram	Hatırlanacak fikir
Batch	Periodic bulk scoring
Online	Request-time inference
Async	Queue-based long jobs
Serverless	Scale-to-zero/event-driven
Staging	Pre-production validation
Canary	Small traffic rollout
Blue/Green	Parallel environments
Shadow	Observe without impact
Autoscaling	Capacity follows load
Readiness	Traffic-ready state
Warm-up	Reduce first-request latency
Load test	Capacity evidence
Fallback	Graceful degradation
Rollback	Return to safe release
p95/p99	Tail latency
RPS	Throughput
SLO	Internal reliability target
Registry	Approved model source
Image digest	Immutable serving runtime
Deployment lineage	Image + model + config

Bu modülden sonra
43. Monitoring bölümünde deployment sonrası data drift, concept drift, model performance, service health ve retraining sinyallerini ayrıntılı olarak izleyeceksin.